```
    @Test
    public void testWithdraw() {
        System.out.println("withdraw");
        int[] score = {1100,5,1100};
        ADiceGame.withdraw(score);
        assertTrue(ADiceGame.withdraw(score));
        score[2] = 6;
        assertFalse(ADiceGame.withdraw(score));
    }

}
```

***Κώδικας 8.9*** *Test για την κλάση AdiceGame*

Καταρχάς να επισημάνουμε τον ρόλο μερικών νέων συναρτήσεων που περιλαμβάνονται στην ADiceGameTest και συναντούμε για πρώτη φορά.

**assertEquals** με δύο παραμέτρους ίδιου τύπου. Ελέγχει τις παραμέτρους και αν δεν είναι ίσες σημειώνει λάθος.

**assertEquals** με τρεις πραγματικές παραμέτρους. Ελέγχει τις δύο πρώτες παραμέτρους για προσεγγιστική ισότητα. Η τρίτη παράμετρος είναι η ανεκτή διαφορά. Στη δική μας περίπτωση χρησιμοποιούμε το EPSILON της εφαρμογής.

**fail** σημειώνει λάθος με μήνυμα την πραγματική παράμετρό της.

**assertTrue** σημειώνει λάθος εφόσον η παράμετρός της είναι ψευδής.

**assertFalse** σημειώνει λάθος εφόσον η παράμετρός της είναι αληθής.

Ας συζητήσουμε τώρα κάποιες από τις συναρτήσεις της ADiceGameTest ώστε να κατανοήσουμε πώς πρέπει να γράφουμε τις συναρτήσεις ελέγχου.

Η συνάρτηση testSum ελέγχει τη συνάρτηση sum της ADiceGame. Γνωρίζουμε ότι έχουμε τρεις πίνακες πραγματικών δηλωμένους στην ADiceGame, τον fair, τον lucky και τον unlucky και πως το άθροισμα των στοιχείων κάθε ενός από αυτούς τους πίνακες πρέπει να είναι ίσο με 1. Μπορούμε να αναπτύξουμε την testSum βασιζόμενοι σε αυτό το δεδομένο. Πράγματι, αν εξετάσετε τον κώδικα της testSum θα διαπιστώσετε περιλαμβάνει διαδοχικές κλήσεις στην sum, μία για κάθε έναν από τους fair, lucky και unlucky. Λαμβάνει το αποτέλεσμα της κλήσης στη μεταβλητή result την οποία συγκρίνει προσεγγιστικά μέσω της assertEquals με το 1. Αν η assertEquals εντοπίσει απόκλιση, θα σημειώσει λάθος.

Η testRoll καλεί την roll 10.000 φορές για κάθε μία κατανομή, δηλαδή για τους πίνακες fair, lucky και unlucky. Αν παραχθεί αριθμός έξω από το όριο 1 έως 6, καλεί την fail και σημειώνει το λάθος. Στο τέλος του βρόγχου καλείται η assertTrue για να ελέγξει τη συνθήκη sumLucky > sumFair && sumFair > sumUnlucky. Πράγματι, γνωρίζουμε πως η κατανομή lucky φέρνει περισσότερα άρτια αποτελέσματα από ότι η fair και η fair περισσότερα από ότι η unlucky. Μετά από 10.000 ρίψεις, οι πιθανότητες λογικά επαληθεύονται. Σε περίπτωση όμως που δεν επαληθευτούν η assertTrue θα σημειώσει λάθος.

Στην testMax δημιουργούμε 1000 πίνακες με τη βοήθεια της constructArray κατά τρόπο που γνωρίζουμε για κάθε πίνακα την τιμή του μέγιστου στοιχείου του. Προσέξτε πως η constructArray ανακατεύει τα στοιχεία του πίνακα που δημιουργεί έτσι ώστε το στοιχείο με τη μέγιστη τιμή να μην βρίσκεται πάντα στην τελευταία θέση του πίνακα. Στη συνέχεια ελέγχουμε αν η τιμή που επιστρέφει η max ανταποκρίνεται στην αναμενόμενη.

Εφόσον έχετε ακολουθήσει τα βήματα έως εδώ με ακρίβεια, το τεστ θα τρέξει επιτυχώς. Καλό είναι όμως να δείτε τι συμβαίνει σε περίπτωση που υπάρχει κάποια αποτυχία στο τεστ.
Μετατρέψτε προσωρινά τις cntOccurances και withdraw της ADiceGame όπως δείχνει ο κώδικας 8.10.

```
    public static int cntOccurances(int[] array, int value) {
        int cnt = 0;
        for (int i = 0; i < array.length; i++) {
            if (array[value] == array[i]) {
                cnt++;
            }
        }
        return cnt;
    }
```

```
    public static boolean withdraw(int[] score) {
        return cntOccurances(score, max(score)) > 1;
    }
```

***Κώδικας 8.10*** *Προσωρινή έκδοση της cntOccurances και withdraw.*

Μετά τη μετατροπή, τρέξτε και πάλι την κλάση ADiceGameTest. Μελετήστε το αποτέλεσμα. Επαναφέρετε τις cntOccurances και withdraw στην προηγούμενη σωστή μορφή τους.

## 8.9 Ασκήσεις προς λύση

### 8.9.1 Τεκμηρίωση, Απολαθοποίηση και Έλεγχος

Δίνεται η κλάση CharArrayLib στον κώδικα 8.11. Πρόκειται για μια βιβλιοθήκη διαχείρισης πινάκων χαρακτήρων. Στην main της κλάσης δημιουργούνται δύο πίνακες χαρακτήρων, ο s1={‘E’.’r’,’i’,’k’} και ο s2={‘s’,’o’,’n’}. Στη συνέχεια δημιουργείται ένας τρίτος πίνακας ως συνένωση των δύο πρώτων. Αμέσως μετά ελέγχεται αν ο πίνακας συνένωση περιλαμβάνει τον s1 και τον s2. Η έξοδος του κώδικα είναι 0 και -1. Αυτό σημαίνει πως ο πίνακας s1 βρέθηκε στον πίνακα συνένωση και αρχίζει στη θέση 0. Ο πίνακας s2 όμως δεν βρέθηκε ενώ περιέχεται στον πίνακα-συνένωση.
Να τεκμηριωθεί και να απολαθοποιηθεί η κλάση και να υλοποιηθεί κατάλληλος κώδικας ελέγχου.

```
public class CharArrayLib {

    static char[] create(String s) {
        return s.toCharArray();
    }

    static int compare(char[] s1, char[] s2) {
        int idx = 0;
        while (idx < s1.length && idx < s2.length) {
            if (s1[idx] > s2[idx]) {
                return 1;
            }
            if (s1[idx] < s2[idx]) {
                return -1;
            }
            idx++;
        }
        return s1.length-s2.length;
    }

    static int compareIgnoreCase(char[] s1, char[] s2) {
        int idx = 0;
        while (idx < s1.length && idx < s2.length) {
            char u1 = Character.toUpperCase(s1[idx]),
                 u2 = Character.toUpperCase(s2[idx]);
            if (u1 > u2) {
                return 1;
            }
            if (u1 < u2) {
                return -1;
            }
            idx++;
        }

        return s1.length - s2.length;
    }
```

```
    static char[] concat(char[] s1, char[] s2) {
        char[] rVal = new char[s1.length + s2.length];
        System.arraycopy(s1, 0, rVal, 0, s1.length);
        System.arraycopy(s2, 0, rVal, s1.length, s2.length);
        return rVal;
    }

    public static boolean equals(char[] tA, char[] tB) {
        if (tA.length != tB.length) {
            return false;
        }
        for (int i = 0; i < tA.length; i++) {
            if (tA[i] != tB[i]) {
                return false;
            }
        }
        return true;
    }

    static char[] subString(char[] s, int from, int to) {
        char[] rVal = new char[to - from];
        int idx = 0;
        for (int i = from; i < to; i++) {
            rVal[idx++] = s[i];
        }
        return rVal;
    }

    static int contains(char[] s, char... cS) {
        if (cS.length > s.length) {
            return -1;
        }

        for (int i = 0; i < s.length - cS.length; i++) {
            char[] sub = subString(s, i, cS.length + i);
            if (equals(sub, cS)) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        char[] s1 = create("Erik"), s2 = create("son");
        char[] con = concat(s1, s2);
        int idx = contains(con, s1);
        System.out.println(idx);
        idx = contains(con, s2);
        System.out.println(idx);
        char[] s4 =create("EriksoN");
        System.out.println(compareIgnoreCase(con,s4));
    }
}
```

***Κώδικας 8.11*** *Βιβλιοθήκη διαχείρισης πινάκων χαρακτήρων*

### 8.9.2 Παιχνίδι με τράπουλα

Τέσσερις φίλοι, ο John, ο Jim, η Mary και η Ann παίζουν χαρτιά. Το παιχνίδι που παίζουν έχει τους ακόλουθους κανόνες: Καταρχάς, ανακατεύεται η τράπουλα, 52 φύλλων (χωρίς μπαλαντέρ). Στον πρώτο γύρο τοποθετούνται τέσσερις κάρτες ανοικτές σε στοίβα στο τραπέζι και μοιράζονται από τέσσερις κάρτες σε κάθε παίκτη. Στους

επόμενους γύρους λαμβάνουν από τέσσερις κάρτες μόνο οι παίκτες. Οι γύροι επαναλαμβάνονται έως ότου εξαντληθεί η τράπουλα.

Σε κάθε γύρο, οι παίκτες ανοίγουν τις κάρτες τους εναλλάξ μια προς μια και τις τοποθετούν στην κορυφή της στοίβας. Αν η κάρτα που άνοιξε ο παίκτης έχει ίδια τιμή με την κάρτα που βρίσκεται στην κορυφή της στοίβας, ο παίκτης κερδίζει τις κάρτες της στοίβας. Επίσης, κερδίζει τις κάρτες της στοίβας εφόσον η κάρτα που άνοιξε είναι βαλές. Όταν εξαντληθεί η τράπουλα, ο παίκτης που έχει τις περισσότερες κάρτες κερδίζει τις τυχόν εναπομείνασες κάρτες της στοίβας. Τέλος, οι παίκτες αθροίζουν τις τιμές των καρτών που έχουν μαζέψει και όποιος συγκεντρώνει το μεγαλύτερο άθροισμα, ανακηρύσσεται νικητής.

Να υλοποιηθεί προσομοίωση μιας παρτίδας παιχνιδιού μεταξύ των τεσσάρων φίλων. Να συνοδευτεί από κατάλληλο κώδικα ελέγχου.

# Βιβλιογραφία

[1] L. B. Wilson and R. G. Clark, Comparative Programming Languages. Pearson Education, 2001.

[2] C. Böhm and G. Jacopini, “Flow diagrams, turing machines and languages with only two formation rules,” Commun. ACM, vol. 9, no. 5, pp. 366–371, May 1966, doi: 10.1145/355592.365646.

[3] “Exception handling,” Wikipedia. Sep. 19, 2021. Accessed: Oct. 15, 2021. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Exception_handling&oldid=1045188202

[4] K. Wiegers and J. Beatty, Software Requirements, 3rd edition. Redmond, Washington: Microsoft Press, 2013.

[5] “javadoc-The Java API Documentation Generator.” https://docs.oracle.com/javase/7/docs/technotes/tools/windows/javadoc.html (accessed Sep. 14, 2021).

[6] “How to Write Doc Comments for the Javadoc Tool.” https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html (accessed Sep. 14, 2021).

# Κεφάλαιο 9

**Σύνοψη**
*Σε αυτήν την ενότητα αναλύεται σε βάθος η έννοια της αναδρομής. Καταρχάς, αναλύεται η έννοια σε γενικότερο πλαίσιο, στη συνέχεια εξηγείται η λειτουργία της μνήμης stack (στοίβα) και μετά παρουσιάζονται οι προγραμματιστικές αναδρομικές τεχνικές, δηλαδή η άμεση και η αμοιβαία αναδρομή. Τέλος, περιλαμβάνεται συζήτηση για τα πλεονεκτήματα και μειονεκτήματα της αναδρομής σε σχέση με τις μη αναδρομικές προσεγγίσεις.*

**Προαπαιτούμενη γνώση**
*Οι θεμελιώδεις τύποι και βασική γνώση του τύπου String. Βασική είσοδος και έξοδος. Η λειτουργία των τελεστών. Οι δομές επιλογής και οι επαναληπτικές δομές. Η διαχείριση των πινάκων. Οι συναρτήσεις, η έννοια και βασική διαχείριση των απροσδόκητων λαθών, οι προσδιοριστές προσπέλασης και η χρήση των πακέτων.*

**Λέξεις κλειδιά**
*Μνήμη stack, αναδρομή, αμοιβαία αναδρομή*

## 9 Αναδρομή

Όταν στον ορισμό μιας έννοιας χρησιμοποιείται η ίδια αυτή έννοια, λέμε πως έχουμε αναδρομικό ορισμό (recursive definition). Χαρακτηριστικό παράδειγμα αναδρομικού ορισμού είναι ο ορισμός της ακολουθίας Fibonacci που είδαμε στην άσκηση 7.8.3. Πιο συγκεκριμένα, αν Fn αναπαριστά τον ν-οστό όρο, η τιμή του υπολογίζεται ως εξής:

$$n = 0 \rightarrow F_n = 0$$
$$n = 1 \rightarrow F_n = 1$$
$$n > 1 \rightarrow F_n = F_{n-1} + F_{n-2}$$

Δηλαδή, αν το n είναι ίσο με το 0 ή το 1, ο ν-οστός όρος ισούται με το n. Για τιμές μεγαλύτερες του 1, ο ν-οστός όρος ισούται με το άθροισμα των δύο προηγούμενων όρων. Επομένως, η τιμή ενός όρου της ακολουθίας Fibonacci ορίζεται με βάση την τιμή δύο όρων της ίδιας ακολουθίας.

Πώς όμως μπορούμε να υπολογίσουμε το F3; Με βάση αυτόν τον ορισμό, F3=F2+F1. Το F1 έχει τιμή 1. Δεν γνωρίζουμε όμως την τιμή του F2. Ωστόσο, γνωρίζουμε πως F2=F1+F0. Επομένως, F2=1 και άρα F3=2.

Ένα άλλο χαρακτηριστικό παράδειγμα είναι ο αναδρομικός ορισμός της δύναμης ενός αριθμού υψωμένου σε εκθέτη. Ας πάρουμε την απλή περίπτωση που η βάση είναι πραγματικός και ο εκθέτης ακέραιος. Σε αυτήν την περίπτωση ο γνωστός μας μη αναδρομικός ορισμός έχει ως εξής;

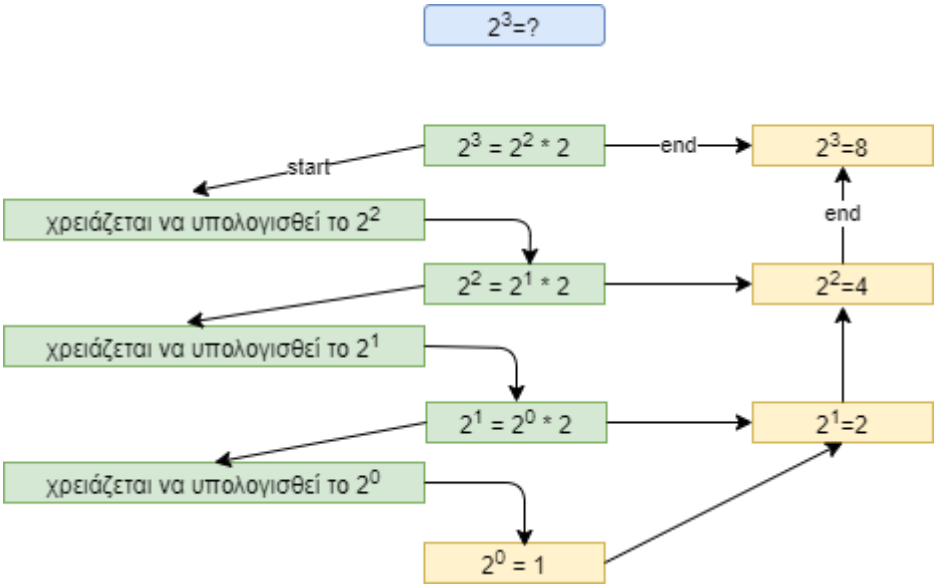
$$a^e = \begin{cases} 1 \; \alpha\nu \; e = 0 \\ a * a * \dots a, e \; \varphi o\rho\acute{\varepsilon}\varsigma \; \alpha\nu \; e > 0 \\ \dfrac{1}{a * a * \dots a, e \; \varphi o\rho\acute{\varepsilon}\varsigma} \; \alpha\nu \; e < 0 \end{cases}$$ [8]

Όμως ισχύει και ο αναδρομικός ορισμός.

$$a^e = \begin{cases} 1, \alpha\nu \; e = 0 \\ a^{e-1} * a, \alpha\nu \; e > 0 \\ \dfrac{1}{a^{-e}}, \alpha\nu \; e < 0 \end{cases}$$

[8] Ας σημειωθεί εδώ πως το $0^0$ δεν ορίζεται. Ωστόσο σε πολλές περιπτώσεις θεωρείται πως $0^0$=1. Πολλές γλώσσες προγραμματισμού συμπεριλαμβανομένων των Java, Javascript και C, C++, υπολογίζουν το $0^0$ ίσο με 1. Για παράδειγμα, αν καλέσετε τη συνάρτηση Math.pow(0, 0), θα διαπιστώσετε πως επιστρέφει 1. Για συμβατότητα θεωρούμε σε αυτό το βιβλίο πως το $0^0 = 1$.

Όπως βλέπουμε στον αναδρομικό ορισμό της δύναμης το $a^e$ ορίζεται ως το $a^{e-1}*a$ όταν το e είναι θετικός και ως $1/a^{-e}$ όταν το e είναι αρνητικός. Πώς θα υπολογίσουμε αναδρομικά την τιμή του $2^3$; Ας το συζητήσουμε με τη βοήθεια του σχήματος 9.1.




***Σχήμα 9.1*** *Ο αναδρομικός υπολογισμός του* $2^3$

Όπως φαίνεται στο σχήμα 9.1, για να υπολογίσουμε το $2^3$ θα πρέπει πρώτα να υπολογίσουμε το $2^2$. Παρόμοια, για να υπολογίσουμε το $2^2$, θα πρέπει πρώτα να υπολογίσουμε το $2^1$ και για να υπολογίσουμε το $2^1$ θα πρέπει να υπολογίσουμε το $2^0$. Το $2^0$ όμως εξ ορισμού είναι ίσο με το 1. Επομένως τώρα εύκολα υπολογίζουμε το $2^1$ που είναι ίσο με 2. Γνωρίζοντας το $2^1$, μπορούμε να υπολογίσουμε το $2^2$ που είναι ίσο με 4. Τέλος, γνωρίζοντας το $2^2$, εύκολα υπολογίζουμε το $2^3$ που είναι ίσο με 8.

Γενικότερα, η αναδρομή αποδομεί ένα πρόβλημα τάξης ν σε ένα ή περισσότερα ίδια προβλήματα τάξης μικρότερης του ν. Η διαδικασία της αποδόμησης επαναλαμβάνεται έως ότου φτάσουμε σε ένα ή περισσότερα απλά προβλήματα με προφανή λύση. Τότε τα αποτελέσματα από τα προφανή προβλήματα αξιοποιούνται για τη λύση των προβλημάτων του αμέσως ανώτερου επιπέδου των οποίων η λύση επίσης συνεισφέρει στη λύση των προβλημάτων του ανώτερου επιπέδου, κοκ, έως ότου φθάσουμε στη λύση του αρχικού προβλήματος.

Στη συνέχεια, θα δούμε την αναδρομή στην Java και θα συζητήσουμε τα πλεονεκτήματα και μειονεκτήματά της. Πρώτα όμως θα δώσουμε μερικές απαραίτητες πληροφορίες για μια περιοχή της μνήμης ενός προγράμματος που ονομάζεται στοίβα (stack).

## 9.1 Η λειτουργία της στοίβας

Για ευκολία θα υποθέσουμε πως κάθε εφαρμογή Java διαθέτει μια μνήμη που ονομάζεται στοίβα[9]. Σύμφωνα με τις προδιαγραφές (specification) της Java [1], η στοίβα μπορεί να είναι σταθερού ή δυναμικά μεταβαλλόμενου μεγέθους. Και στις δύο περιπτώσεις, η στοίβα μπορεί να γεμίσει με δεδομένα. Στην πρώτη περίπτωση, αφού καλυφτεί το προκαθορισμένο μέγεθός της και στη δεύτερη αφού καταλάβει όλη τη διαθέσιμη δυναμική μνήμη. Αν το πρόγραμμά μας απαιτεί επιπλέον μνήμη στοίβας, τότε έχουμε αδυναμία συνέχισης της εκτέλεσης του προγράμματος. Επομένως, η εξάντληση της στοίβας είναι ένα πολύ σοβαρό πρόβλημα που πάντα πρέπει να λαμβάνει ο προγραμματιστής υπόψη του.

Ας δούμε όμως τι αποθηκεύεται στη στοίβα και ποια μπορεί να είναι η πιθανή αιτία εξάντλησής της. Στον κώδικα 9.2 παρουσιάζουμε μια απλή μελέτη περίπτωσης.

```
public class StackFunctionality {
```

[9] Στην πραγματικότητα, στις πολυνηματικές (multithreading) εφαρμογές, διαμορφώνεται μια στοίβα για κάθε νήμα.

```
    static void a() {
        String id = "static function a";
        System.out.println("start " + id);
        String end = "end";
        b();
        System.out.println(end + " " + id);
    }

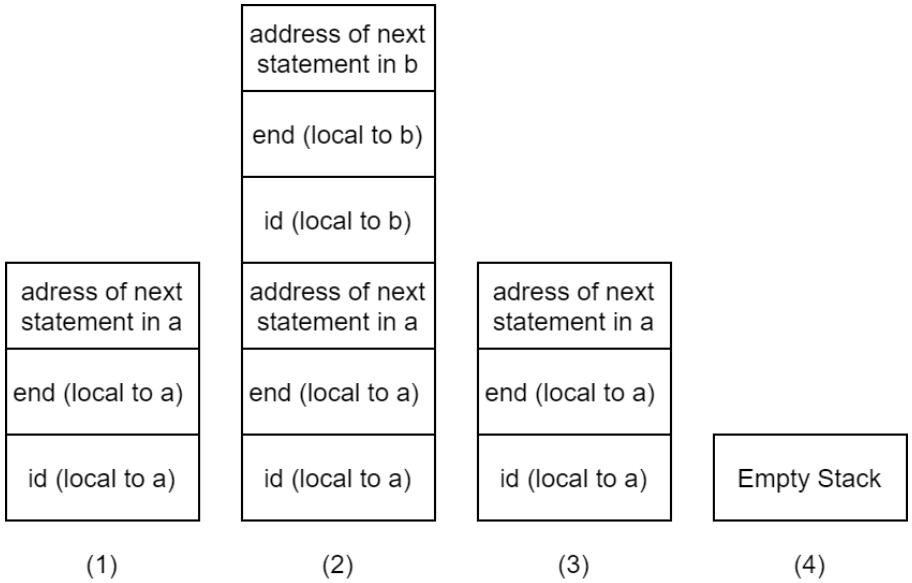
    static void b() {
        String id = "static function b";
        System.out.println("start " + id);
        String end = "end";
        c();
        System.out.println(end + " " + id);
    }

    static void c() {
        String id = "static function c";
        System.out.println("start " + id);
        String end = "end";
        System.out.println("c is running");
        System.out.println(end + " " + id);
    }

    public static void main(String[] args) {
        a();
    }
}
```

***Κώδικας 9.1*** *Περίπτωση μελέτης λειτουργίας της στοίβας*

Όπως βλέπουμε στον κώδικα 9.1, η συνάρτηση a καλεί την b και αυτή με τη σειρά της, την c. Όταν όμως η c θα ολοκληρωθεί, θα πρέπει η b να συνεχίσει. Από ποιο σημείο όμως; Μα από τη γραμμή της a που έπεται της κλήσης της c. Το ίδιο θα συμβεί όταν ολοκληρωθεί η εκτέλεση της b, δηλαδή θα πρέπει να συνεχίσει η a. Προφανώς, από τη γραμμή του κώδικα που έπεται της κλήσης της b. Η συνάρτηση a προσπελαύνει τις μεταβλητές end και id. Όμως, η b διαθέτει δικές της τέτοιες μεταβλητές. Η προσπέλαση με τα αναγνωριστικά end και id μέσα στην b αναφέρεται στις τοπικές μεταβλητές της b. Αντίθετα, οι τοπικές μεταβλητές της a είναι απροσπέλαστες κατά την εκτέλεση της b. Επομένως, είναι φανερό πως η κλήση συνάρτησης μέσα από άλλη συνάρτηση απαιτεί μια ειδική διαχείριση. Η διαχείριση αυτή βασίζεται στη στοίβα.
Η στοίβα είναι μια Last In First Out (LIFO) δομή, δηλαδή ό,τι μπαίνει τελευταίο στη στοίβα βγαίνει πρώτο ή αλλιώς ό,τι εισάγεται στη στοίβα, τοποθετείται στην κορυφή της και ό,τι εξάγεται, λαμβάνεται από την κορυφή της.




***Σχήμα 9.2*** *Η στοίβα κατά την εκτέλεση της main του κώδικα 9.1*

Ας δούμε όμως πώς λειτουργεί η στοίβα κατά την εκτέλεση του κώδικα 9.1. Για απλοποίηση, στην περιγραφή αυτή παραλείπουμε τις κλήσεις πέραν των κλήσεων της a, της b και της c. Καταρχάς, έχουμε την κλήση της a. Η a κατανέμει τις τοπικές μεταβλητές της και προχωρά στην εκτέλεσή της μέχρι του σημείου που καλεί την b. Πριν ακριβώς ξεκινήσει η b, οι τοπικές μεταβλητές της a αποθηκεύονται στη στοίβα όπως δείχνει το σχήμα 9.2-1. Όπως φαίνεται σε αυτό το σχήμα, εκτός από τις τοπικές μεταβλητές, στη στοίβα αποθηκεύεται και η διεύθυνση του κώδικα από την οποία θα συνεχιστεί η εκτέλεση της a όταν η b θα έχει τερματίσει. Στη συνέχεια εκτελείται η b μέχρι του σημείου που καλείται η c. Πριν την κλήση της c, οι τοπικές μεταβλητές της b και η διεύθυνση της γραμμής που έπεται της κλήσης της c, επίσης αποθηκεύονται στη στοίβα. Έτσι, η στοίβα διαμορφώνεται όπως δείχνει το σχήμα 9.2-2. Με τον τερματισμό της c, η b εξάγει τις απαραίτητες πληροφορίες από την κορυφή της στοίβας και συνεχίζει την εκτέλεσή της, οπότε η στοίβα διαμορφώνεται όπως δείχνει το σχήμα 9.2-3. Παρόμοια, με τον τερματισμό της b, η a εξάγει από τη στοίβα τις πληροφορίες που την αφορούν και οι οποίες μετά την εξαγωγή των πληροφοριών της b, βρίσκονται στην κορυφή της στοίβας. Έτσι, η a συνεχίζει απρόσκοπτα τη λειτουργία της, ενώ η στοίβα έχει αδειάσει (σχήμα 9.2-4).

Επομένως, κάθε φορά που μια συνάρτηση καλείται από μία άλλη, στη στοίβα τοποθετούνται διάφορες πληροφορίες. Άρα είναι δυνατόν αν έχουμε πολύ μεγάλο βάθος κλήσεων, η στοίβα να μην διαθέτει άλλο χώρο. Ωστόσο, κάτι τέτοιο είναι απίθανο να συμβεί μέσα από μη αναδρομικές κλήσεις. Μπορεί όμως να συμβεί από αναδρομικές κλήσεις. Πόσες φορές θα καλέσει μία συνάρτηση τον εαυτό της, εξαρτάται από το μέγεθος του προβλήματος που επιχειρεί να επιλύσει.

## 9.2 Αναδρομικές συναρτήσεις

Σύμφωνα με τα όσα έχουμε πει, είναι εύκολο να υποθέσουμε πως μια συνάρτηση είναι αναδρομική όταν καλεί τον εαυτό της. Προσέξτε τη συνάρτηση endless στον κώδικα 9.2.

```
static void endless(int i) { //Κώδικας 9.2
    System.out.println(i);
    endless(i + 1);
}
```

***Κώδικας 9.2*** *Αναδρομική συνάρτηση χωρίς συνθήκη ελέγχου*

Καταρχάς, είναι φανερό πως πρόκειται για αναδρομική συνάρτηση. Αν καλέσουμε endless(0), τότε η συνάρτηση θα εμφανίσει 0 και στη συνέχεια θα καλέσει endless(1). Η endless(1) θα εμφανίσει 1 και θα καλέσει endless(2). Παρόμοια, η endless(2) θα εμφανίσει 2 και θα καλέσει endless(3). Η αναδρομική κλήση θα συνεχιστεί στην ουσία μέχρις ότου γεμίσει η στοίβα και το πρόγραμμα πέσει.

Επομένως, κάθε αναδρομική συνάρτηση πρέπει να περιλαμβάνει μια συνθήκη ελέγχου της αναδρομικής κλήσης. Με άλλα λόγια, μια αναδρομική συνάρτηση πρέπει να περιλαμβάνει κάποιο κώδικα που κάτω από κάποιες συνθήκες θα τερματίζει την αναδρομική κλήση.

```
static void withEnd(int i) { //Κώδικας 9.3
    System.out.println(i);
    if (i < 3) {
        withEnd(i + 1);
    }
}
```

***Κώδικας 9.3*** *Απλή αναδρομή με συνθήκη τερματισμού*

Όπως φαίνεται στον κώδικα 9.3, η withEnd είναι επίσης αναδρομική. Ωστόσο, η αναδρομική κλήση γίνεται μόνο εφόσον το i είναι μικρότερο του 3. Για παράδειγμα, αν καλέσουμε withEnd(0), η συνάρτηση θα εμφανίσει 0, στη συνέχεια ελέγχει το i, το βρίσκει μικρότερο του 3 καλεί την endless(1). Με την ίδια διαδικασία, τυπώνει 1, μετά 2, καλεί και withEnd(3), εμφανίζει και το 3 και μετά βρίσκει ότι η έκφραση i<3 είναι ψευδής και σταματά την αναδρομική κλήση. Σε αυτήν την περίπτωση μπορούμε να πούμε ότι η withEnd δεν πέφτει. Ισχύει όμως αυτό γενικευμένα; Η απάντηση είναι όχι. Όπως αναφέραμε και προηγουμένως, εξαρτάται από το μέγεθος του προβλήματος. Για παράδειγμα, αν καλέσουμε withEnd(Integer.MIN_VALUE), θα έχουμε τόσες αναδρομικές κλήσεις ώστε η συνάρτηση θα πέσει πολύ πριν κληθεί με παράμετρο 3.

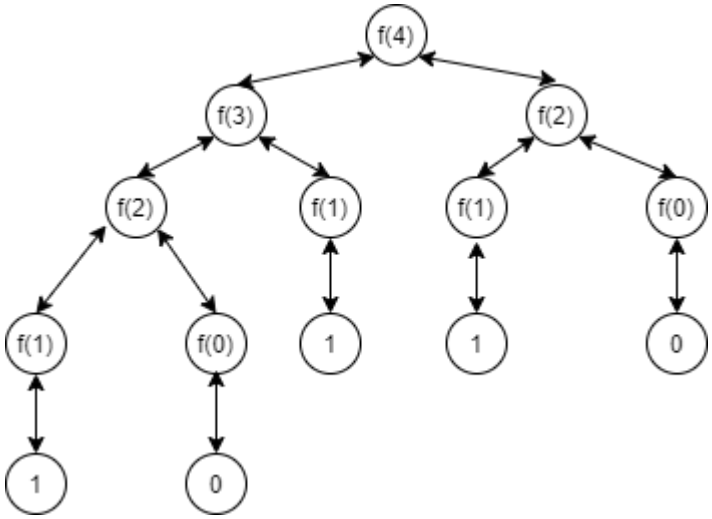
Αυτό είναι και το βασικό μειονέκτημα της αναδρομής. Οποιαδήποτε αναδρομική συνάρτηση μπορεί να αντιμετωπίσει πρόβλημα χωρητικότητας της στοίβας ανάλογα με το μέγεθος του προβλήματος που επιχειρεί να αντιμετωπίσει.

Στον κώδικα 9.4 παραθέτουμε την αναδρομική υλοποίηση υπολογισμού του ν-οστού όρου της ακολουθίας Fibonacci.

```
static int fibonacci(int n) { //Κώδικας 9.4
    if (n < 0) {
        throw new RuntimeException();
    }
    if (n <= 1) {
        return n;
    } else {
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
}
```

***Κώδικας 9.4*** *Αναδρομική υλοποίηση υπολογισμού το ν-οστού όρου της ακολουθίας Fibonacci*

Στην ακολουθία Fibonacci δεν ορίζονται όροι αρνητικής τάξης. Έτσι αν η συνάρτηση κληθεί με αρνητικό όρισμα παράγει εξαίρεση. Στη συνέχεια, ο κώδικας 9.4 υλοποιεί ακριβώς τον ορισμό της ακολουθίας που δόθηκε στην αρχή της ενότητας 9. Ας δούμε πώς λειτουργεί η συνάρτηση όταν κληθεί με όρισμα 4. Στο σχήμα 9.3, για απλοποίηση, η συνάρτηση συμβολίζεται με f αντί για fibonacci.




***Σχήμα 9.3*** *Κλήσεις και τιμές επιστροφής της fibonacci*

Όπως βλέπουμε στο σχήμα 9.3, καταρχάς έχουμε κλήση της f(4). Σύμφωνα με τον κώδικα 9.4, η κλήση αυτή θα επιστρέψει f(3)+f(2). Επομένως, καλείται η f(3) και η f(2) όπως δείχνει και το σχήμα 9.3. Στη συνέχεια, ο υπολογισμός της f(3) απαιτεί την κλήση της f(2) και f(1) και ο υπολογισμός της f(2) απαιτεί την κλήση f(1) και f(0). Σε αυτό το βάθος τρεις κόμβοι του δέντρου, οι f(1), f(1) και f(0), υπολογίζονται χωρίς περαιτέρω αναδρομική κλήση. Αντίθετα ο κόμβος f(2) απαιτεί την κλήση των f(1) και f(0). Κάθε τερματικός κόμβος επιστρέφει την τιμή του στο αμέσως προηγούμενο επίπεδο. Έτσι το f(1) υπολογίζεται ως 1, το f(0) ως 0, το f(2) ως 1, το f(3) ως 2 και το f(4) ως 3.

Ας σημειωθεί πως η περίπτωση κατά την οποία το n είναι μικρότερο ή ίσο του 1 ονομάζεται βασική περίπτωση (base case). Γενικότερα, στην αναδρομή, βασική περίπτωση ονομάζεται η περίπτωση κατά την οποία υπάρχει προφανής λύση που ο υπολογισμός της δεν απαιτεί αναδρομική κλήση.

Το παράδειγμα που θα δούμε τώρα είναι απλό, ωστόσο ιδιαίτερα κρίσιμο για την κατανόηση του μηχανισμού αναδρομικών κλήσεων.

```
static void prt(int i) { // Κώδικας 9.5 prt(1) 123321
    System.out.print(i);
    if (i < 3) {
        prt(i + 1);
    }
    System.out.print(i);
}
```

***Κώδικας 9.5*** *Συνέχεια της συνάρτησης μετά την αναδρομική κλήση*

Προσέξτε τη συνάρτηση prt του κώδικα 9.5. Τι εμφανίζει κατά την άποψή σας η κλήση prt(3); Συχνά, λαμβάνω λανθασμένη απάντηση σε αυτήν την ερώτηση. Πολλοί φοιτητές στο στάδιο της εκμάθησης της αναδρομής θεωρούν πως η prt(0) θα εμφανίσει 0123. Για να το δούμε βήμα προς βήμα. Όταν αρχίσει η εκτέλεση της prt(0), είναι προφανές πως καταρχάς θα εμφανίσει 0. Στη συνέχεια, ελέγχει τη συνθήκη i<3 την οποία βρίσκει αληθή και εκτελεί το μπλοκ της if. Με την ολοκλήρωση του μπλοκ της if, ο έλεγχος επανέρχεται στην κλήση prt(0). Η συνάρτηση εξάγει την τιμή της i από τη στοίβα και προχωράει στην εκτέλεση της τελευταίας εντολής της στην οποία εμφανίζει και πάλι το 0. Άρα, μέχρι εδώ έχουμε υπολογίσει μια έξοδο ως εξής: 0[Έξοδος από το μπλοκ της if, με i==0]0. Στο μπλοκ της if καλείται καταρχάς, η prt(1) η οποία λειτουργεί ακριβώς με τον ίδιο τρόπο, δηλαδή το αποτέλεσμά της μπορεί να περιγραφεί ως 1[Έξοδος από το μπλοκ της if, με i==1]1 ενώ το συνολικό αποτέλεσμα μέχρι στιγμής είναι 01[Έξοδος από το μπλοκ της if, με i==1]01. Συνεχίζοντας με αυτόν τον τρόπο φτάνουμε στο τελικό αποτέλεσμα που είναι 01233210.

Το κλειδί εδώ είναι να προσέξουμε το εξής: Η prt(0) καλεί κάποια στιγμή την prt(1). Αυτό όμως δεν σημαίνει πως η prt(0) έχει ολοκληρωθεί. Όταν η αναδρομική κλήση θα ολοκληρωθεί, η prt(0) θα συνεχίσει από την επόμενη γραμμή κώδικα από όπου έγινε η κλήση prt(1).

Ας δούμε τώρα την αναδρομική συνάρτηση δύναμης με βάση πραγματικό και εκθέτη ακέραιο.

```
static double power(double b, int e) { //Κώδικας 9.6
    if (e > 0) {
        return power(b, e - 1) * b;
    }
    if (e < 0) {
        return 1 / power(b, -e);
    }
    return 1;
}
```

***Κώδικας 9.6*** *Αναδρομική υλοποίηση του υπολογισμού δύναμης*

Όταν έχουμε διαθέσιμο έναν αναδρομικό ορισμό, είναι εύκολο να υλοποιήσουμε την αντίστοιχη αναδρομική συνάρτηση. Θυμηθείτε τον αναδρομικό ορισμό της δύναμης στην αρχή της ενότητας 9 και συγκρίνετέ τον με τον κώδικα 9.6. Θα διαπιστώσετε σημαντική ομοιότητα. Στην ουσία, πρόκειται για απλή μεταγραφή από τη γλώσσα των μαθηματικών στη γλώσσα του προγραμματισμού.

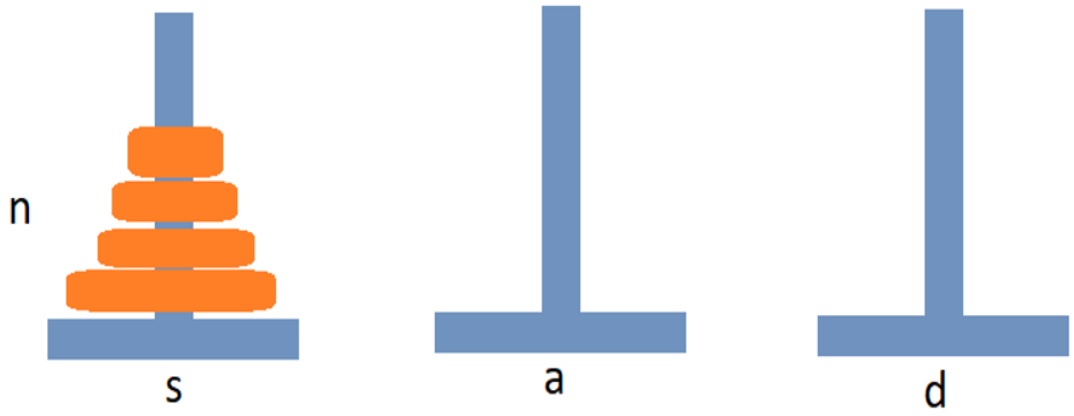
Η συνάρτηση power ελέγχει τον εκθέτη e, αν είναι θετικός επιστρέφει power(b, e-1)*b, δηλαδή ακριβώς ό,τι λέει και ο μαθηματικός ορισμός. Αν πάλι ο εκθέτης είναι αρνητικός, επιστρέφει 1/power(b,-e). Τέλος, εφόσον ο εκθέτης είναι 0, δεν θα γίνουν οι αναδρομικές κλήσεις που προβλέπονται για θετικό και αρνητικό εκθέτη, οπότε η συνάρτηση θα επιστρέψει 1.

## 9.3 Οι πύργοι του Ανόι

Οι πύργοι του Ανόι (towers of Hanoi) είναι ένα μαθηματικό πρόβλημα που χρησιμοποιείται ευρύτατα για την εκμάθηση της αναδρομής στον προγραμματισμό. Η λύση του φαίνεται αρχικά αρκετά περίπλοκη σε σχέση με τις λύσεις των προβλημάτων που έχουμε δει ως τώρα. Ωστόσο, αν σκεφτεί κανείς με τον κατάλληλο τρόπο, θα διαπιστώσει πως η αναδρομική λύση απαιτεί περιορισμένο κώδικα και τελικά αποδεικνύεται απλούστερη της μη αναδρομικής.

### 9.3.1 Περιγραφή του προβλήματος

Έχουμε τρεις στύλους, τον s από το source (προέλευση), τον d από το destination (προορισμός) και τον a από το auxiliary (βοηθητικός). Στον στύλο s είναι τοποθετημένοι n δίσκοι όπως φαίνεται στο σχήμα 9.4. Πιο συγκεκριμένα, οι δίσκοι έχουν διαφορετική διάμετρο και είναι τοποθετημένοι έτσι ώστε ένας δίσκος με μεγαλύτερη διάμετρο δεν βρίσκεται ποτέ επάνω από έναν δίσκο με μικρότερη διάμετρο.




***Σχήμα 9.4*** *Οι πύργοι του Ανόι*

Το ζητούμενο είναι να μεταφέρουμε τους δίσκους από τον στύλο s στον στύλο d χρησιμοποιώντας ως βοηθητικό τον στύλο a. Η μετακίνηση όμως θα πρέπει να γίνει με τους εξής κανόνες:

- Κάθε φορά μπορούμε να μετακινήσουμε έναν μόνο δίσκο από έναν στύλο σε έναν άλλο.